Problem Statement:

Implement DEAP (Distributed Evolutionary Algorithms) using Python.

In [3]:
```
from deap import base, creator, tools
import random
```

1. Create types

In [4]:
```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
```

2. Toolbox

In [5]:
```
toolbox = base.Toolbox()
```

Attribute (random numbers)

In [6]:
```
toolbox.register("attr_float", random.random)
```

Individual & population

In [7]:
```
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, n=5)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

3. Evaluation function

In [8]:
```
def evaluate(individual):
    return sum(individual),

toolbox.register("evaluate", evaluate)
```

Operators

In [9]:
```
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)
```

4. Main algorithm

In [10]:
```
def main():
    pop = toolbox.population(n=10)

    # Evaluate initial population
    for ind in pop:
        ind.fitness.values = toolbox.evaluate(ind)

    # Evolution
    for gen in range(10):
        offspring = toolbox.select(pop, len(pop))
        offspring = list(map(toolbox.clone, offspring))

        # Crossover
        for c1, c2 in zip(offspring[::2], offspring[1::2]):
            if random.random() < 0.5:
                toolbox.mate(c1, c2)
                del c1.fitness.values, c2.fitness.values

        # Mutation
        for mutant in offspring:
            if random.random() < 0.2:
                toolbox.mutate(mutant)
                del mutant.fitness.values

        # Re-evaluate
        invalid = [ind for ind in offspring if not ind.fitness.valid]
        for ind in invalid:
            ind.fitness.values = toolbox.evaluate(ind)

        pop[:] = offspring
```

```
    # Best individual
    best = tools.selBest(pop, 1)[0]
    print("Best Individual:", best)
    print("Best Fitness:", best.fitness.values)
```

In [11]:
```
main()
```

```
Best Individual: [2.842341729119222, 1.8359472339572942, 1.3073561035515087, 0.7541874750600248, 1.8379659003269877]
Best Fitness: (8.577798442015037,)
```